Commonly Used SQL Commands for Interviews

1. SELECT

Description: Retrieve data from a table.

Example:

```
SELECT * FROM employees;
```

2. WHERE

Description: Filter records based on conditions.

Example:

```
SELECT * FROM employees WHERE department = 'HR';
```

3. JOIN

Description: Combine rows from two or more tables based on related columns.

Example:

```
SELECT employees.name, departments.department_name
FROM employees
INNER JOIN departments ON employees.department_id = departments.id;
```

4. GROUP BY

Description: Group rows sharing a property and perform aggregate functions.

Example:

```
SELECT department, COUNT(*) AS num_employees
FROM employees
GROUP BY department;
```

5. HAVING

Description: Filter grouped records (used with GROUP BY).

Example:

```
SELECT department, COUNT(*) AS num_employees
FROM employees
GROUP BY department
HAVING COUNT(*) > 5;
```

6. INSERT INTO

Description: Insert new records into a table.

Example:

```
INSERT INTO employees (name, department) VALUES ('John Doe', 'Sales');
```

7. UPDATE

Description: Modify existing records in a table.

Example:

```
UPDATE employees SET salary = 50000 WHERE id = 101;
```

8. DELETE

Description: Remove records from a table.

Example:

```
DELETE FROM employees WHERE id = 101;
```

9. DISTINCT

Description: Return only distinct (unique) values.

Example:

```
SELECT DISTINCT department FROM employees;
```

10. ORDER BY

Description: Sort results in ascending or descending order.

```
Example:

SELECT * FROM employees ORDER BY salary DESC;
```

11. LIMIT

Description: Limit the number of records returned.

```
Example:

SELECT * FROM employees LIMIT 10;
```

12. ALTER TABLE

Description: Modify an existing table (e.g., add or drop columns).

```
Example:

ALTER TABLE employees ADD COLUMN date_of_birth DATE;
```

13. DROP TABLE

Description: Remove a table from the database.

```
Example:

DROP TABLE employees;
```

14. CREATE INDEX

Description: Create an index on one or more columns for faster searching.

```
Example:

CREATE INDEX idx_name ON employees (name);
```

15. LIKE

Description: Search for a specified pattern in a column.

```
Example:

SELECT * FROM employees WHERE name LIKE 'J%';
```

16. BETWEEN

Description: Filter records within a range of values.

```
Example:

SELECT * FROM employees WHERE salary BETWEEN 30000 AND 50000;
```

17. IN

Description: Filter records that match a list of values.

```
Example:

SELECT * FROM employees WHERE department IN ('HR', 'Sales');
```

18. IS NULL

Description: Check for NULL values.

```
Example:

SELECT * FROM employees WHERE date_of_birth IS NULL;
```

19. CASE

Description: Perform conditional logic in a query.

```
Example:

SELECT name,
       CASE
           WHEN salary > 50000 THEN 'High'
```

```
           WHEN salary BETWEEN 30000 AND 50000 THEN 'Medium'
           ELSE 'Low'
       END AS salary_level
FROM employees;
```

20. EXISTS

Description: Check if a subquery returns any results.

```
Example:
SELECT name
FROM employees e
WHERE EXISTS (
    SELECT 1
    FROM orders o
    WHERE o.employee_id = e.id
);
```

21. UNION

Description: Combine the results of two or more SELECT queries.

```
Example:
SELECT name FROM employees
UNION
SELECT name FROM contractors;
```

22. CAST

Description: Convert one data type to another.

```
Example:
```

```
SELECT CAST(salary AS DECIMAL(10, 2)) FROM employees;
```

23. SUBSTRING

Description: Extract a substring from a string.

```
Example:

SELECT SUBSTRING(name, 1, 3) FROM employees;
```

24. CONCAT

Description: Concatenate two or more strings.

```
Example:

SELECT CONCAT(first_name, ' ', last_name) FROM employees;
```

25. COALESCE

Description: Return the first non-NULL value.

```
Example:

SELECT COALESCE(address, 'Not Available') FROM employees;
```